

如何实现单片机系统的低功耗

肖建云

2007.9

主题

- 一、如何设计一个低功耗的单片机系统
- 二、Silabs MCU在低功耗方面的优势
- 三、Silabs MCU在低功耗方面的设计方法

一、如何设计一个低功耗的单片机系统

问题：单片机系统的功耗是否只是由单片机的功耗决定？

以单片机为核心构成的系统，其系统的总能耗是由单片机能耗及其外围电路能耗共同构成。为了降低整个系统的功耗，除了要降低单片机自身的运行功耗外，还要降低外围电路的功耗。

如何设计低功耗单片机系统？

要设计一个低功耗的单片机系统，需要从硬件和软件两方面入手。

1、硬件设计

1.1 选用尽量简单的CPU内核

在选择CPU内核时切忌一味追求性能。选择的原则应该是“够用就好”。8位机够用，就没有必要选用16位机。一般来说，单片机的运行速度越快，功耗也越大。一个复杂的CPU集成度高、功能强，但片内晶体管多，总漏电流大，即使进入STOP状态，漏电流也变得不可忽视；而简单的CPU内核不仅功耗低，成本也低。

1.2 选用低电压供电的系统

低电压供电可以大大降低系统的工作电流。目前单片机从与TTL兼容的5 V供电降低到3.3 V、3 V、2 V乃至1.8 V供电，降低单片机的供电电压可以有效降低其功耗。供电电压降低也是未来单片机发展的一个重要趋势。

1.3 选择带有低功耗模式的系统

低功耗模式指的是系统的**IDLE**、**STOP**和**SUSPEND**模式。处于这类模式下的单片机功耗将大大小于运行模式下的功耗。

项 目 模 式	CPU	时钟 (内 部)	片上数 字外设	片上模 拟外设	内部 寄存 器和 RAM	功耗	退出 条件	PC指针
IDLE	Inactive	active	active	active	保持	小于正常模式的功耗 (C8051F41 0 5mA)	任何使能的中断或复位	0000或从进入IDLE模式的指令的下一条指令开始执行
STOP	Inactive	Inactive	Inactive	使能但不运行或用户设定关闭	RAM保持, 内部寄存器在唤醒时被复位	功耗非常低 (C8051F41 0 15nA)	内部或外部的复位	0000
SUSPEND	Inactive	Inactive	使能但不运行或用户设定关闭	使能但不运行或用户设定关闭	保持	功耗低 (C8051F41 0 15nA)	唤醒事件或内部和外部的复位	0000或从进入SUSPEND模式的指令的下一条指令开始执行

Professional Channel Partner, Professional Support

1.4 选择合适的时钟方案

时钟的选择对于系统功耗相当敏感，有两方面的问题要注意：

(1) 系统总线频率应当尽量低。

单片机内部的总电流消耗分为：运行电流和漏电流。

单片机集成度越高，环境温度越高，漏电流也越大。

单片机的运行电流几乎和其时钟频率成正比。降低时钟频率，就可以有效降低单片机的功耗。

(2) 时钟方案

是否使用锁相环，使用内部振荡器还是外部振荡器。

现代单片机普遍使用锁相环技术，使单片机的时钟频率可以由程序控制。单片机使用外部较低的振荡器，通过软件控制，系统时钟可以在一个很宽的范围内调整，得到比较高的总线时钟。使用锁相环会带来额外的功耗。单就时钟方案来讲，使用外部晶振且不使用锁相环是功率消耗最小的一种。

有的单片机带有内部时钟，也可使用外部时钟。这可以根据实际系统的需要使用双时钟：一个高速时钟和一个低速时钟。处理事件时使用高速时钟，空闲时使用低速时钟。这钟双时钟系统可以有效地降低功耗。

2、应用软件开发

应用软件开发对于一个低功耗系统的重要性常常被人们忽略。一个重要的原因是，软件上的缺陷并不像硬件那样容易发现，同时也没有一个严格的标准来判断一个软件的低功耗特性。尽管如此，设计者如果能尽量将应用的低功耗特性反映在软件中，就可以避免那些“看不见”的功耗损失。

2.1 用“中断”代替“查询”

在没有要求低功耗的场合，程序使用中断方式还是查询方式并不重要。但在要求低功耗场合，这两种方式相差甚远。使用中断方式，**CPU**可以什么都不做，甚至可以进入等待模式或停止模式；而查询方式下，**CPU**必须不停地访问**I/O**寄存器，这会带来很多额外的功耗。

2.2 用“宏”代替“子程序”

子程序调用的入栈出栈操作，要对**RAM**进行两次操作，会带来更大的功耗。宏在编译时展开，**CPU**按顺序执行指令。使用宏，会增加程序的代码量，但对不在乎程序代码量大的应用，使用宏无疑会降低系统的功耗。

2.3 尽量减少CPU的运算量

减少CPU的运算工作量，可以有效地降低CPU的功耗。减少CPU运算的工作可以从很多方面入手：

- (1) 用查表的方法替代实时的计算；
- (2) 不可避免的实时计算，算到精度够了就结束，避免“过度”的计算；
- (3) 尽量使用短的数据类型，例如，尽量使用字符型的8位数据替代16位的整型数据，尽量使用分数运算而避免浮点数运算等。

2.4 让I/O模块间歇运行

(1) 不用的I/O模块或间歇使用的I/O模块要及时关掉，以节省电能。

(2) 不用的I/O引脚要设置成输出或设置成输入，用上拉电阻拉高。

二、Silabs MCU 在低功耗方面 的优势

1、Silabs MCU供电电压低

Silabs MCU供电电压为：2.0~5.25V。供电电压低可以有效降低整个单片机系统的功耗。

2、Silabs MCU有多种低功耗模式

Silabs MCU有多种低功耗模式：**Idle**模式、**Suspend**模式和**Stop**模式。用户可以根据实际情况选择所需要的低功耗模式。其中在**Suspend**模式和**Stop**模式下的功耗可以达到nA级。

–40 to +85 °C, 50 MHz System Clock unless otherwise specified. Typical values are given at 25 °C

Parameter	Conditions	Min	Typ	Max	Units
Core Supply Current with CPU inactive (not accessing Flash)	V_{DD} = 2.0 V:				
	Clock = 32 kHz	—	10	TBD	μA
	Clock = 200 kHz	—	22	—	μA
	Clock = 1 MHz	—	0.15	TBD	mA
	Clock = 25 MHz	—	2.8	—	mA
	Clock = 50 MHz	—	5	TBD	mA
	V_{DD} = 2.5 V:				
	Clock = 32 kHz	—	11	TBD	μA
	Clock = 200 kHz	—	30	—	μA
	Clock = 1 MHz	—	0.21	TBD	mA
Core Supply Current (suspend)	Oscillator not running, VDD = 2.5 V	—	150	TBD	nA
Core Supply Current (shutdown)	Oscillator not running, VDD = 2.5 V	—	150	TBD	nA

Professional Channel Partner, Professional Support

3、Silabs MCU有多种时钟方案供选择

Silabs MCU内置振荡器有高速震荡模式和低速震荡模式可供选择。每种模式下的频率又有多种选择。而且还可以外接振荡器。**更重要的是，在MCU运行中，这些时钟模式可以实时切换。**这很方便客户进行低功耗控制。例如：在处理数据时，系统运行在高速状态；空闲时运行在低速状态。

4、高速实时的中断响应

Silabs MCU响应中断的时间非常快，一般只需要5个系统时钟周期。中断响应速度快，CPU花费在等待方面的时间少，这可以节省不少的等待功耗。

5、灵活的I/O设置

Silabs MCU的I/O口资源丰富，配置灵活。有三种配置方式：漏极开路、推拉输出和弱上拉方式。用户可以根据实际需要通过对相关寄存器的设置来禁止或使能这些方式。其中将端口配置成漏极开路方式是最省电的方式。

另外，**Silabs MCU**片上没有用到的其他外设可以通过软件来关闭。

总之，根据项目的要求，灵活运用**Silabs**的各种低功耗特性，通过软件的控制，就可以很好地实现低功耗的要求。

6、使用每MIPS功耗来衡量MCU的低功耗性能是相对比较准确。

比如执行一个需要10K条指令的任务,甲MCU的工作电流为3mA,速度为10MIPS,则甲MCU需要工作1mS完成该任务,消耗 $3\text{mA} \times 1\text{mS} \times V_{cc}$,然后甲MCU就可以进入低功耗模式了.而乙MCU的工作电流为1mA,速度为2MIPS,则乙MCU需要工作5mS完成,这样乙MCU完成该任务的消耗为 $1\text{mA} \times 5\text{mS} \times V_{cc}$.

电流大但是速度快的MCU可能更省电!
***Silabs MCU*使用每MIPS功耗来评估也是性能最好的!!!!**

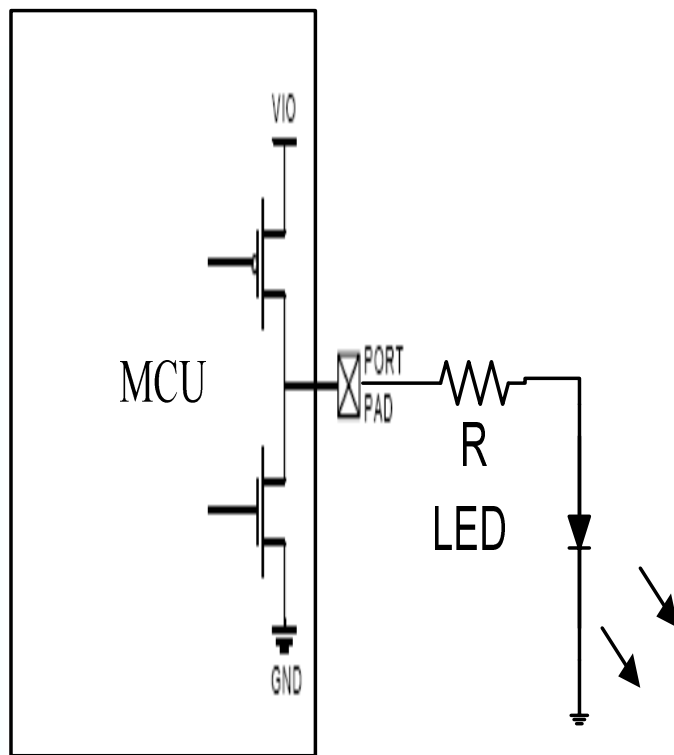
三、**Silabs MCU**在低功耗方面的 设计方法

一般来说，MCU的运行的速度越高，供电电压越高，功耗也就越高。要降低单片机系统的功耗，就要降低单片机系统的供电电压，降低MCU运行的频率。

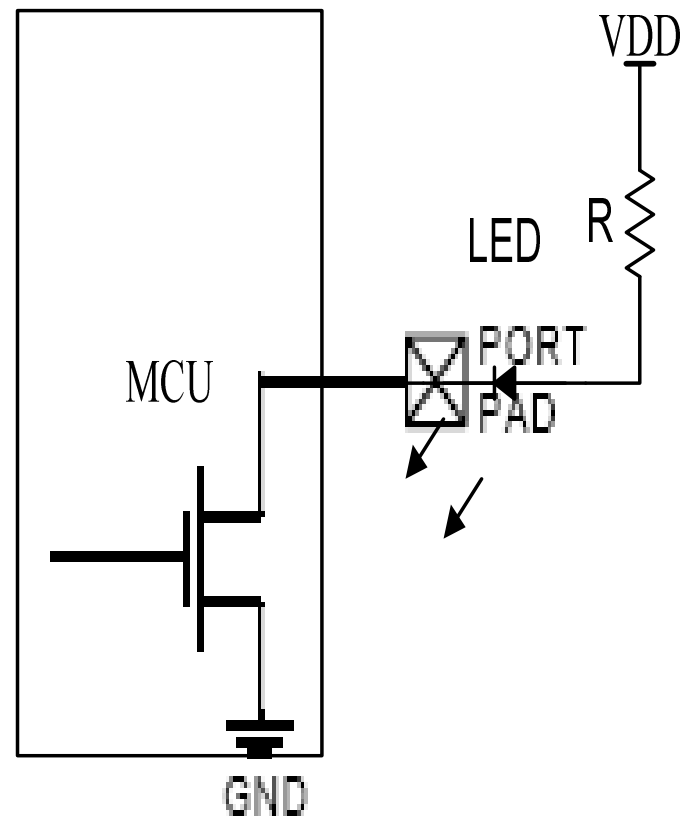
对于**Silabs MCU**来说，内部弱上拉的电阻大概**100K**千欧左右，所以对于功耗要求很严的场合，选择适当的电阻，使用外部上拉会更省电。

要驱动外部**LED**，**Silabs MCU I/O**端口能承受**10mA**的拉电流，而**LED**一般**5mA**就可以驱动了，所以尽量选用**I/O**模块的**Push-Pull**方式来驱动**LED**（如图A所示），如果采用**Open-Drain**方式（如图B所示），当关闭**LED**时，**I/O**端口内部的漏电流要比图A所示方式大。

根据实际情况选择合适的低功耗模式，在进入低功耗模式时关闭不用的**I/O**模块和片上的模拟及数字外设，可以进一步降低系统功耗。



A



B

案例分析：电子运动表

客户要做一个电子运动表，使用电池做供电电源，要求平均功耗不超过**200uA**。电子运动表是间歇工作的：当收到数据时激活，快速处理数据；当空闲时进入休眠状态，来降低功耗。

客户使用的是**C8051F333**。

下面我们来看看**C8051F333**的电气参数。

1、正常模式，CPU从Flash取指令

Digital Supply Current—CPU Active (Normal Mode, fetching instructions from Flash)

I _{DD} (Note 3)	V _{DD} = 3.6 V, F = 25 MHz	—	10.7	11.7	mA
	V _{DD} = 3.0 V, F = 25 MHz	—	7.8	8.3	mA
	V _{DD} = 3.0 V, F = 1 MHz	—	0.38	—	mA
	V _{DD} = 3.0 V, F = 80 kHz	—	31	—	μA

2、Idle模式，CPU停止工作

Digital Supply Current—CPU Inactive (Idle Mode, not fetching instructions from Flash)

I_{DD} (Note 3)	$V_{DD} = 3.6\text{ V}, F = 25\text{ MHz}$	—	4.8	5.2	mA
	$V_{DD} = 3.0\text{ V}, F = 25\text{ MHz}$	—	3.8	4.1	mA
	$V_{DD} = 3.0\text{ V}, F = 1\text{ MHz}$	—	0.20	—	mA
	$V_{DD} = 3.0\text{ V}, F = 80\text{ kHz}$	—	16	—	μA

Professional Channel Partner, Professional Support

光从上面两个表的数据，我们会得出什么结论？

您的答案肯定是不行！

因为要满足系统的运行速度，又要满足低功耗，
 $\geq 1\text{MHz}$ 的频率，功耗都要超过客户的要求。

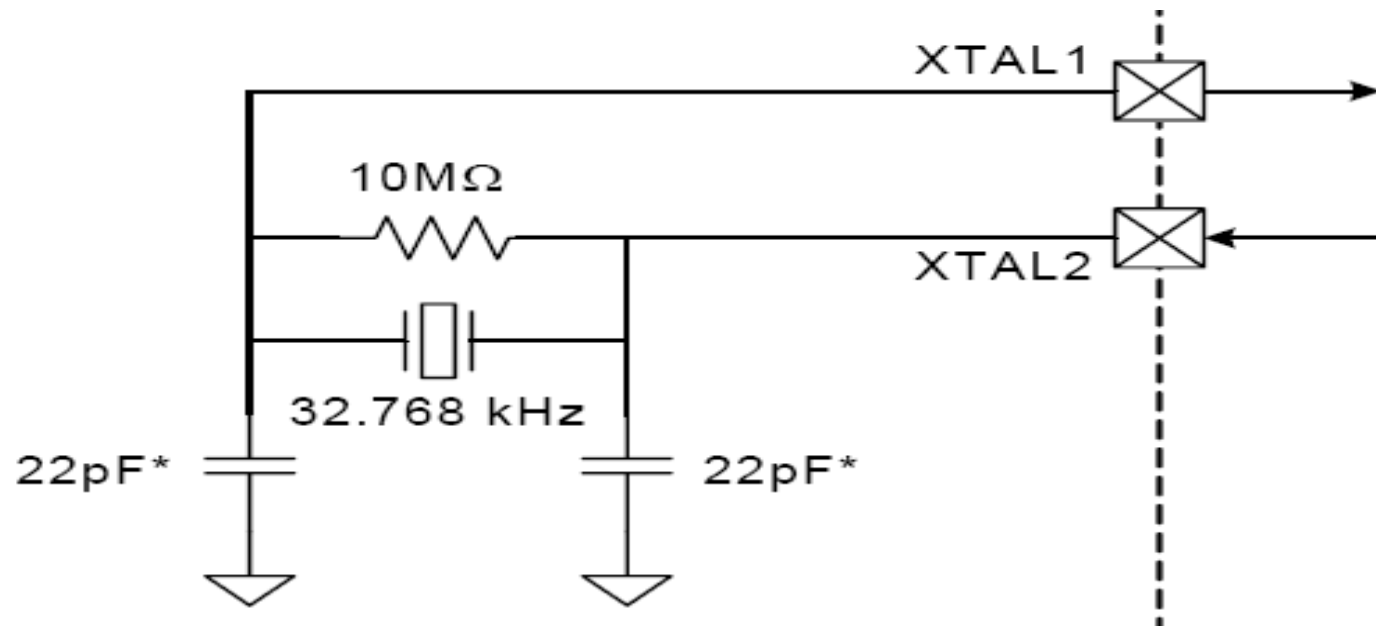
真的是这样吗？

正确答案是：在不降低MCU运行速度（MCU处理数据时的运行频率是24.5MHz）的情况下，客户使用C8051F313成功实现了低功耗的要求，功耗在150uA以下。

鱼和熊掌兼得！

这是怎样实现的？

客户使用了内外两种晶振。工作时使用内部高速晶振**24.5MHZ**，空闲时切换到外部低速晶振**32.768KHZ**，并且进入**Idle**模式。并且把没有用到的外设全部关闭。



Professional Channel Partner, Professional Support

就是这么简单！

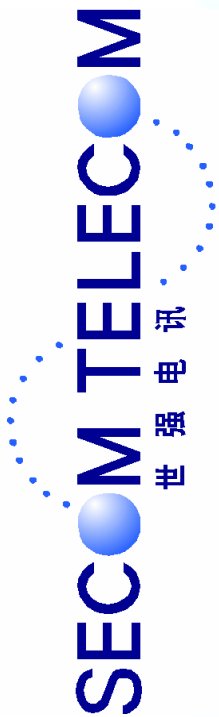
始终牢记功耗是一个系统的问题！

单片机系统的功耗

是由MCU和其外围电路的功耗共同决定的！

认真细致地分析单片机系统的工作特点，灵活运用单片机的低功耗特性，您会发现实现单片机系统的低功耗是那么的容易！

有时候，单片机系统的低功耗就是从硬件和软件的设计中“抠门”出来的！



Thank you!